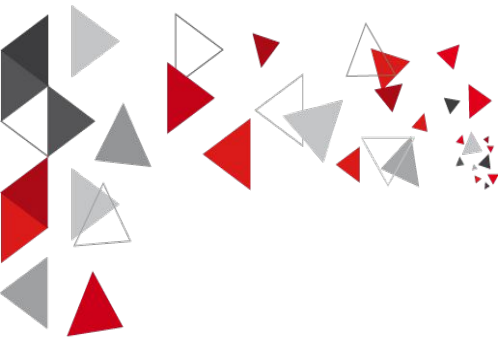
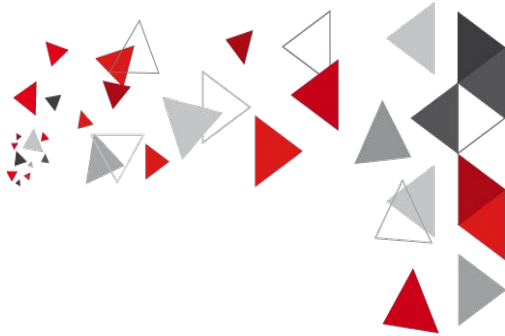


esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

# Conception Orienté Objet et Programmation Java

## Chapitre 8: Les interfaces

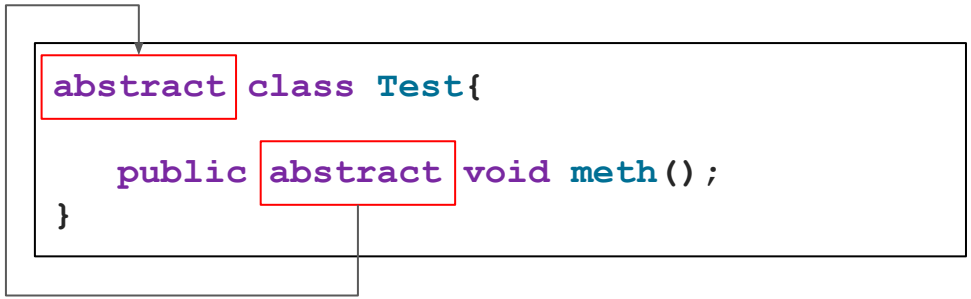
## Objectifs du chapitre



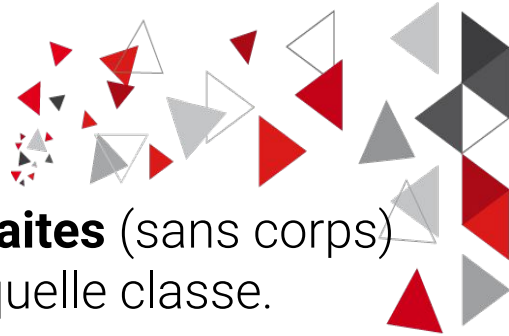
- Se familiariser avec le concept d’interfaces
- Appliquer le concept d’héritage en utilisant des interfaces
- Manipuler une interface

# Rappel : Classe et Méthode Abstraite

- On définit une méthode abstraite comme étant une méthode n'ayant pas de corps.
- Une fois on déclare une méthode abstraite dans une classe A, cette classe devient abstraite aussi.
- Une méthode abstraite ne peut pas être déclarée **static ou private ou final**.



# Les interfaces



En Java, une **interface** est un ensemble de **méthodes abstraites** (sans corps) et de **constantes**, qui peut être implémentée par n'importe quelle classe.

Elle définit un ensemble de fonctionnalités qu'une classe peut offrir **sans définir** comment ces fonctionnalités sont implémentées.

En d'autres termes, une interface peut être considérée comme un contrat que toute classe qui l'implémente **doit respecter**.

Pour déclarer une interface en Java, on utilise le mot-clé "**interface**" suivi du nom de l'interface.

On dit qu'une classe **implémente** une interface.

```
interface MyInterface{

    float PI = 3.14f;

    void meth1();
    int meth2(String str);
}
```



- Toutes les méthodes au sein d'une interface sont par nature **abstraites**.
- Tous les membres d'une interface sont automatiquement **publiques**.
- Tout attribut déclarés dans un interface est une **constante statique**.

```
interface MyInterface{

    float PI = 3.14f; //static, final et public sont optionnels

    void meth1(); //abstract et public sont optionnels
    int meth2(String str);
}
```

**Attention : On ne peut pas modifier le niveau de visibilité des membres d'une interface.**

Une classe qui implémente une interface **doit** fournir une implémentation (le corps des méthodes) pour **toutes les méthodes** définies dans l'interface.

Une classe qui implémente une interface, et n'implémente pas toute ses méthodes doit être déclarée comme **abstraite**.

```
class MyClass implements MyInterface{

    void meth1(){
        //Code d'implémentation de la méthode 1
    }
    int meth2(String str){
        //Code d'implémentation de la méthode 2
    }
}
```



Si une classe contient que des méthodes abstraites, on peut la convertir en une interface.

```
abstract class MyClass{

    public abstract void meth1();
    public abstract int meth2(String str);
    public abstract float meth3(int x);
}
```

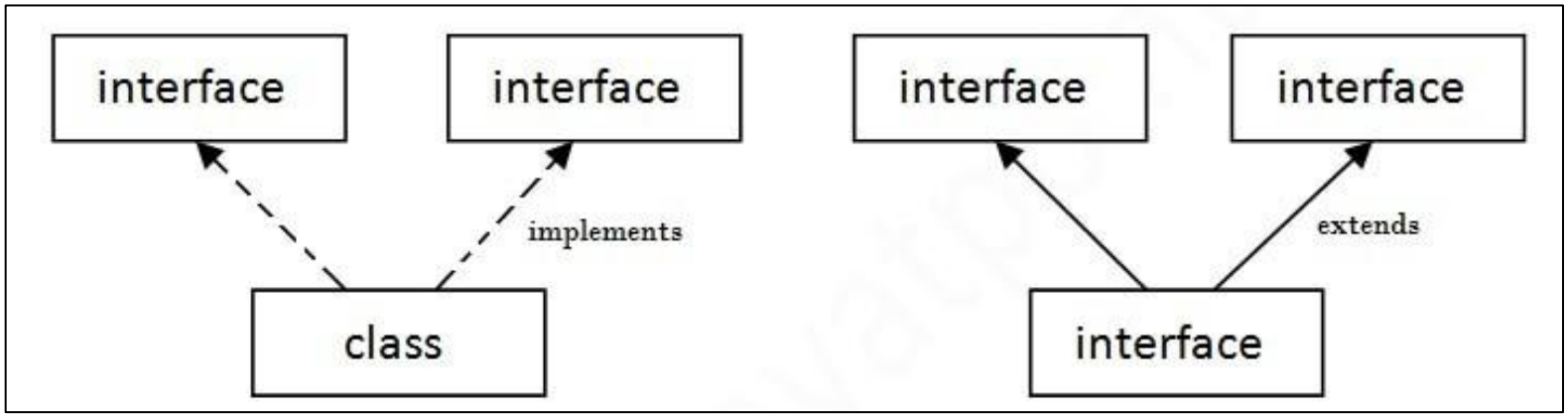
Si une classe contient que des méthodes abstraites, on peut la convertir en une interface.

```
interface MyInterface{

    void meth1();
    int meth2(String str);
    float meth3(int x);
}
```

Une classe peut implémenter **plusieurs** interfaces à la fois, et une interface peut hériter de **plusieurs** autres interfaces à la fois.

**Attention : En JAVA l'héritage multiple pour une classe n'est pas permis, donc les interfaces répondent à cette problématique.**



# Les interfaces : Héritage (exemple 1)

```
interface Math1{

    int add(int x,int y);
    int multiply(int x,int y);
}
```

```
interface Math2{

    int divide(int x,int y);
    int subtract(int x,int y);
}
```

```
class MyClass implements Math1, Math2{

    public int add(int x,int y){return x + y;}
    public int multiply(int x,int y){return x * y;}
    public int divide(int x,int y){return x / y;}
    public int subtract(int x,int y){return x - y;}
}
```

# Les interfaces : Héritage (exemple 2)

```
interface Math1 extends Math2{

    int add(int x,int y);
    int multiply(int x,int y);
}
```

```
interface Math2{

    int divide(int x,int y);
    int subtract(int x,int y);
}
```

```
class MyClass implements Math1{

    public int add(int x,int y){return x + y;}
    public int multiply(int x,int y){return x * y;}
    public int divide(int x,int y){return x / y;}
    public int subtract(int x,int y){return x - y;}
}
```

## Les méthodes par défaut



Avec les méthodes par défaut (**"default methods"**), il est possible de fournir une implémentation commune pour une méthode dans une interface, ce qui évite la duplication de code dans chaque classe qui l'implémente.

```
interface Math1{

    int add(int x,int y);
    int multiply(int x,int y);

    default int divide(int x,int y){
        return x / y;
    }
}
```

## Les méthodes par défaut

Une méthode par défaut **peut être redéfinie**, mais ce n'est **pas obligatoire** puisqu'elle n'est **pas abstraite**.

```
class MyClass implements Math1{

    public int add(int x,int y){return x + y;}
    public int multiply(int x,int y){return x * y;}

    public int divide(int x,int y){
        int z = Math1.super.divide(x,y);
        return z / 2;
    }
}
```

## Les méthodes par défaut : Avantages



L'utilisation des méthodes par défaut peut :

- Simplifier le code en réduisant la duplication de code dans les classes qui implémentent une interface.

- Mettre à jour plus facilement une interface existante sans avoir à modifier toutes les classes qui l'implémentent.

Il est possible aussi déclarer une méthode statique dans une interface, ces méthodes fonctionnent exactement de la même façon que celles portées par les classes.

```
interface Math1{

    int add(int x,int y);
    int multiply(int x,int y);

    static int subtract(int x,int y){
        return x - y;
    }
}
```

**Attention : Une méthode dans une interface ne peut pas être statique et par défaut à la fois.**



## Les méthodes par défaut :

- Peut être appelé qu'à partir d'une instance d'une classe qui implémente l'interface.
- Peut être redéfinie

## Les méthodes statiques :

- L' appel à une méthode statique se fait via le nom de l'interface.
- Ne Peut pas être redéfinie

```
public static void main(String[] args) {

    MyClass c = new MyClass();
    c.divide();

    MyInterface.subtract();
}
```

# abstract class vs interface



| Abstract class | Interface |
| --- | --- |
| N'ont pas de restrictions sur la visibilité des attributs et des méthodes | Tous les constantes et les méthodes sont publiques |
| Peut avoir des constructeurs qui peuvent être exécutés lors de l'instanciation d'une classe fille | Peut contenir des méthodes par défaut (avec implémentation) |
| Peut contenir des méthodes concrètes (avec implémentation) | |
| Une classe peut étendre une seule classe abstraite. | Une classe peut implémenter plusieurs interfaces. |

## Les interfaces génériques

Une **interface générique** est une interface **paramétrée** par **un ou plusieurs types**, ce qui permet de créer des classes et des méthodes flexibles et réutilisables avec différents types de données.

Pour passer un paramètre à une interface, nous spécifions un seul caractère majuscule pour le nom du paramètre de type générique ("E", "T", "K", etc.).

```
interface Math1<T>{

    T add(T x,T y);
    T multiply(T x,T y);
}
```

```
interface Math2<T,K>{

    K add(T x,T y);
    K multiply(T x,T y);
}
```

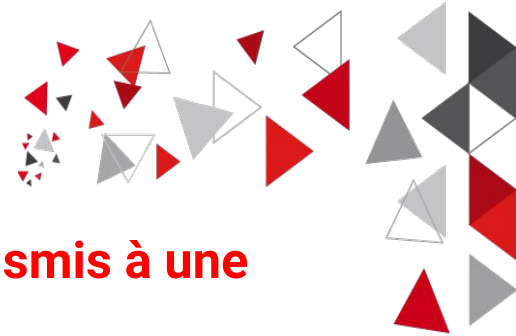
Cela permet d'avoir différentes implémentations des mêmes méthodes avec un type de données différent pour chaque classe.

```
class IntegerMath implements Math1<Integer>{

   public Integer add(Integer x,Integer y){return x + y;}
    public Integer multiply(Integer x,Integer y){return x * y;}
}
```

```
class FloatMath implements Math1<Float>{

   public Float add(Float x,Float y){return x + y;}
    public Float multiply(Float x,Float y){return x * y;}
}
```



**Attention : Seuls les types de Référence peuvent être transmis à une interface.**

```
class IntegerMath implements Math2<Integer,Float>{

   public Float add(Integer x,Integer y){return x + y;}
    public Float multiply(Integer x,Integer y){return x * y;}
}
```

# Enum

En Java, les **énumérations (enum)** sont des types de “classes” spéciales qui permettent de définir un ensemble fixe de constantes ou de valeurs énumérées.

Les enums sont souvent utilisés pour définir des types de données avec un nombre limité de choix, comme les jours de la semaine, les mois de l'année, les types de véhicules, etc.

# Enum

Les enums sont déclarés en utilisant le mot-clé **enum**, suivi d'une liste de constantes (valeurs énumérées) entre accolades.

```
//déclaration d'un enum de String
public enum MathOperator{ ADD, SUBSTRACT, DIVIDE, MULTIPLY }

class Maths{

     public Float calculate(Float x,Float y, MathOperator operator){
          if(operator.equals(MathOperator.ADD))
               return x + y;

          if(operator.equals(MathOperator.MULTIPLY))
               return x * y;

          return 0f;
     }
}
```

# Merci pour votre attention

